Detecting House Cricket Courtship Signals in Audio Files Using CNN

GO, AJ

***Abstract*—Manually reviewing audio recordings to identify cricket courtship is a time-consuming and tedious task, motivating the need for an automated approach. This work presents a machine learning pipeline that detects cricket courtship signals in audio recordings by converting short clips into spectrograms and classifying them using a convolutional neural network. Cricket courtship is composed of two distinct sounds called ticks and pulses. They are distinguished from other cricket sounds and background noise, after which detected ticks are combined into full courtship events using a simple deterministic rule based on tick timing. The model was evaluated using leave-one-session-out cross-validation to account for variation between recording sessions, achieving high balanced accuracy, and the effect of key preprocessing parameters, such as spectrogram window size, was analyzed to identify the best-performing configuration.**

I. Introduction

This work is part of a broader research program investigating cricket behaviour, a part of which is also investigating if courtship signals in male house crickets are condition dependent. As part of that goal, manually reviewing months of laboratory audio recordings to identify courtship signals is a time-consuming and tedious task. In this paper, we present software that automatically detects cricket courtship signals in audio recordings, eliminating the need to manually inspect spectrograms and label each event by hand. This enables more efficient downstream analysis of courtship behavior.

II. Related Work

A few studies have investigated the meaning and success of cricket courtship based on characteristics such as frequency and length [1]. Other, more closely related studies have researched the classification of sounds from different animals [2]. This work presents a combination of the two.

III. Data

A. Data Collection

A wide range of data is collected as part of the project, including feeding records (what crickets were fed and how often) and digital recordings of crickets — audio, vibration, and camera footage — captured under controlled laboratory conditions. For the purpose of detecting courtship signals, we focus only on audio recordings. Figure 1 shows a clip from a recording with a frequency range from 0 Hz to 24,000 Hz.

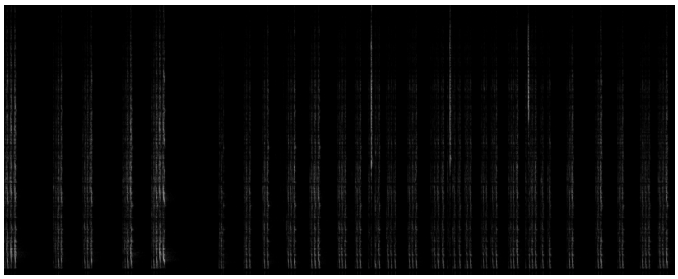

Fig. 1. A clip showing different cricket chirps. Short sounds called ticks at higher frequencies, pulses and other cricket chirps and noise appearing across the whole frequency range.

Recordings were saved as stereo WAV files with a sample rate of 48,000 Hz; however, only the mono audio channel was used for this work, as the second channel captured vibration data. A few months of recordings have been collected form November 2025 to April 2026, of which data from 7 different days was used for machine learning as described in next chapters.

B. Data Preprocessing

In this section, we describe the steps taken to produce clips used as model input for classification.

1) Defining Courtship Events: A courtship event is defined as a sequence of ticks - high frequency signals and pulses - low frequency signals between the ticks. When ticks are at least 3 seconds apart, they are considered to be part of different courtship. This definition is deterministic and straight forward to implement; the remaining challenge is therefore reduced to recognizing individual ticks. To achieve this, we produced clips (a short spectrogram image) to train a model to distinguish between ticks and other sounds, including pulses and background noise. Once the model is able to classify these two types of clips, we apply it to unseen recordings by classifying each fixed-length clip across the recording. Combining the classified ticks into courtship events is then a trivial, deterministic problem.

2) Spectrogram Generation: Starting from raw WAV files, we generated spectrograms rather than using the raw waveform data directly. Frequency content plays a major role in recognizing ticks, and this information is not directly accessible from the raw waveform. While it would be possible to train a model directly on raw audio, doing so would omit information critical to classification. We therefore chose to represent each clip as a spectrogram instead.

Clips were extracted based on a labeled dataset in which ticks and noise events were annotated as single points in time, marking when each event occurred. Since these labels correspond to a single instant rather than a start and end time, extracted clips were widened around each labeled point to ensure the full event was captured.

To determine appropriate spectrogram parameters, we analyzed a sample of 100 clips. Chirp duration ranged from 30 ms to 50 ms, with an average length of 40.5 ms, and chirp frequency content ranged from 5 kHz to 24 kHz. Based on this, all spectrograms were cropped to a frequency range of 5–24 kHz as shown in Figure 2.

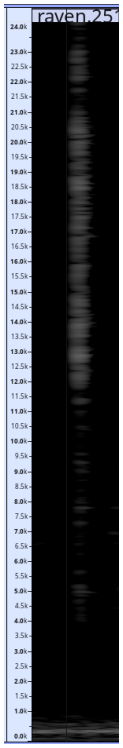

Fig. 2. Spectrogram of a single tick, which spans from 5kHz to 24 kHz frequency range.

The hop size, which determines how many audio samples are represented by a single spectrogram column, was chosen so that an average-length chirp would span approximately 16 pixels in width:

$$\text{hop size} = \frac{T_{\text{chirp}} \times f_s}{W_{\text{target}}} = \frac{0.0405 \times 48000}{16} \approx 121.5$$

This was rounded to a hop size of 128, a power of two, to ensure full ticks were captured while keeping computation efficient. Since tick labels mark only a single point in time rather than precise boundaries, chirps were not always perfectly centered within their 16-pixel window. To account for this, clips were widened symmetrically in both directions, producing a final clip width of 24 pixels, ensuring that full chirps were reliably captured even when off-center. This resulted in a final clip duration of:

$$T_{\text{clip}} = \frac{W_{\text{final}} \times \text{hop size}}{f_s} \times 1000 = \frac{24 \times 128}{48000} \times 1000 \approx 64 \text{ ms}$$

The FFT window size N used to compute the spectrogram is discussed later, as its value was selected based on downstream classification performance (see Section V).

3) Dataset and Clip Format: In total, we generated 5,570 clips to train the model: 2,706 tick clips and 2,864 noise clips. To allow the model to process clips efficiently, each clip was exported in NumPy `.npy` format, containing a 2D array of 32-bit floating-point values.

IV. Methodology

In this section, we describe how the model was trained and optimized, along with the metrics used to evaluate it.

A. Prediction Task

The goal of our model is to perform binary classification of clips, distinguishing ticks from pulses, alarm calls and noise. We focus on maximizing balanced accuracy, since a false tick detection is just as undesirable as a missed tick — precision is not weighted as more important than recall, or vice versa.

B. Data Splitting Strategy

Recordings were made on different days, with different crickets recorded each day, meaning tick characteristics varied between days. To avoid data leakage [3], we used leave-one-session-out (LOSO) cross-validation: in each fold, clips from all execpt one days were used for training, and clips from the remaining day were held out for validation.

C. Model

We use a convolutional neural network (CNN) to classify spectrogram clips. CNNs are well suited to this task because spectrograms are effectively 2D images, where relevant patterns (such as the frequency band and pulse structure of a tick) appear as localized visual shapes rather than being tied to specific pixel positions. CNNs exploit this through convolutional filters, which detect the same pattern regardless of where it appears in the clip, and through weight sharing, which keeps the number of parameters small relative to a fully connected network operating on the same input size. This makes CNNs both effective at capturing the spatial structure of a chirp and efficient to train on a dataset of this size.

D. Metrics

We evaluate model performance using accuracy, balanced accuracy, precision, recall, and F1 score.

*1) **Precision**:* Precision measures, of all clips predicted as ticks, how many were actually ticks:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Low precision indicates the model frequently misclassifies noise as tick (false positives).

*2) **Recall**:* Recall measures, of all actual ticks, how many the model correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Low recall indicates the model frequently misses ticks (false negatives).

*3) **Accuracy**:* Accuracy measures the proportion of all clips, ticks and noise combined, that were correctly classified:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy can be misleading under class imbalance, since a model biased toward the majority class can achieve high accuracy without reliably detecting the minority class.

*4) **Balanced Accuracy**:* Balanced accuracy averages the model's performance on each class independently, rather than across all clips combined:

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2} \quad (1)$$

$$= \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right) \quad (2)$$

This directly reflects our evaluation priority: **sensitivity** captures how well ticks are detected, while **specificity** captures how well noise is correctly rejected (i.e., not falsely flagged as tick). By weighting both equally, balanced accuracy penalizes false positives and false negatives symmetrically, and also guards against any class imbalance that may occur within individual LOSO folds.

*5) **F1 Score**:* F1 score is the harmonic mean of precision and recall:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

Unlike the arithmetic mean, the harmonic mean penalizes cases where precision and recall are highly imbalanced.

E. Generating Clips

1) STFT Parameter — Window Size: As part of our experiments, we tested different STFT window sizes when generating spectrograms. Window size determines the trade-off between time resolution and frequency resolution: a larger window improves frequency resolution but reduces the precision with which chirp onset and offset are localized in time, while a smaller window improves time resolution at the cost of coarser frequency detail. The window size ultimately selected, and its effect on model performance, is discussed in Section V.

TABLE I
MEAN LOSO CROSS-VALIDATION METRICS ACROSS WINDOW SIZES (HOP SIZE = 128, FREQUENCY RANGE 5–24 KHZ, CLIP WIDTH = 24 PIXELS).

Win. Size	Accuracy	Balanced Acc.	Precision	Recall	F1
128	0.9899	0.9901	0.9935	0.9846	0.9888
256	**0.9896**	**0.9905**	**0.9893**	**0.9875**	**0.9881**
512	0.9866	0.9878	0.9863	0.9844	0.9850
1024	0.9848	0.9853	0.9898	0.9778	0.9833

V. RESULTS

With respect to the previous chapter's description, we trained a model using clips generated with spectrograms at different STFT window sizes. Table I shows the mean LOSO cross-validation metrics obtained for each window size.

Overall, all four window sizes achieve strong performance, with balanced accuracy ranging from 0.9853 to 0.9905. A clear trend emerges as window size increases beyond 256: balanced accuracy, accuracy, and F1 score all decline, with the 1024-sample window performing worst across every metric except precision. This is consistent with the time–frequency resolution trade-off inherent to the STFT: larger windows improve frequency resolution but reduce time resolution, which is disadvantageous here since ticks are short in duration (averaging 40.5 ms) and must be precisely localized in time.

The 256-sample window achieves the highest balanced accuracy (0.9905), marginally outperforming the 128-sample window (0.9901). Given that balanced accuracy is our primary evaluation metric, we select the 256-sample window size for the final model. However, the difference between the 128 and 256 window sizes is small across all metrics, indicating the model is not highly sensitive to this parameter within that range, and either would be a reasonable choice.

VI. CONCLUSION

We presented a CNN-based pipeline for automatically detecting cricket courtship song from spectrograms, achieving a balanced accuracy of 0.9905 under leave-one-session-out cross-validation. As future work, we plan to explore wavelet-based representations as an alternative to spectrograms, which may offer improved time resolution for short ticks.

REFERENCES

[1] R. BALAKRISHNAN and G. S. POLLACK, "Recognition of courtship song in the field cricket,teleogryllus oceanicus," *Animal Behaviour*, vol. 51, no. 2, pp. 353–366, 1996. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0003347296900342

[2] S. J. Harrison, I. R. Thomson, C. M. Grant, and S. M. Bertram, "Calling, courtship, and condition in the fall field cricket, Gryllus pennsylvanicus," *PLoS One*, vol. 8, no. 3, p. e60356, 2013.

[3] L. Sasse, E. Nicolaisen-Sobesky, J. Dukart, S. B. Eickhoff, M. Götz, S. Hamdan, V. Komeyer, A. Kulkarni, J. M. Lahnakoski, B. C. Love, F. Raimondo, and K. R. Patil, "Overview of leakage scenarios in supervised machine learning," *Journal of Big Data*, vol. 12, no. 1, May 2025. [Online]. Available: http://dx.doi.org/10.1186/s40537-025-01193-8